

金融計算期末報告

子主題(A)

財三 B 江慶栩

目錄：

壹、 題目

貳、 研究動機

參、 資料來源

肆、 步驟與分析

（一）假設股價為 Geometric Brownian motion，以模擬方法探討比較歐式選擇權買權（European call option）模擬價格和波動（ σ ）之關係，並由此得到結論。

1. 目的
2. 程式
3. 結果
4. 證據
5. 結論

（二）（a）增加比較不同波動（ σ ）所設定的亂數種子（random seed）固定與不固定比較，並由此得到結論。

1. 目的
2. 程式
3. 結果
4. 證據
5. 結論

（三）（b）增加比較使用 moment matching 與不使用 moment matching 之結果比較，並由此得到結論。

1. 目的
2. 程式
3. 結果
4. 證據
5. 結論

（四）（c）結果以 streamlit 視覺化互動網頁呈現。

1. 目的
2. 程式

3. 結果
4. 證據
5. 結論

伍、總結

壹、題目

假設股價為 Geometric Brownian motion，以模擬方法探討比較歐式選擇權買權（European call option）模擬價格和波動（ σ ）之關係，並由此得到結論。

- （a） 增加比較不同波動（ σ ）所設定的亂數種子（random seed）固定與不固定比較，並由此得到結論。
- （b） 增加比較使用 moment matching 與不使用 moment matching 之結果比較，並由此得到結論。
- （c） 結果以 streamlit 視覺化互動網頁呈現。
- （d） 加入完整結論。

貳、研究動機

過去在修習財務工程、投資學與財務管理等課程時，我已學過選擇權、風險與報酬、資產價格變動以及 Black-Scholes 模型等相關概念。其中，波動率是影響選擇權價格的重要因素，也是在實務上最難準確掌握的參數之一。因此，本子主題 A 以 European call option 為研究對象，假設股價服從 Geometric Brownian Motion，並透過 Monte Carlo 模擬分析不同波動率下的買權價格變化，再與 Black-Scholes 理論價格進行比較。此外，本研究也進一步觀察固定與不固定種子碼、以及是否使用 moment matching 對模擬結果穩定性的影響，最後以 streamlit 互動網頁呈現，希望能將過去課程中學到的理論基礎，延伸到實際的金融計算與模擬分析中。

參、資料來源

本研究的資料來源主要來自金融計算課程之上課內容與範例程式，包含 Geometric Brownian Motion、Black-Scholes 選擇權定價模型、Monte Carlo 模擬 random seed 設定與 moment matching 方法。本文所使用的選擇權價格並非實際市場資料，而是依據課堂模型假設，透過 Python 程式自行模擬產生，並以 Black-Scholes 理論價格作為比較基準。

肆、步驟與分析

（一）題目：

假設股價為 Geometric Brownian motion，以模擬方法探討比較歐式選擇權買權（European call option）模擬價格和波動（ σ ）之關係，並由此得到結論。

1. 目的

本程式的目的，是在假設股價服從 Geometric Brownian Motion 的情況下，使用蒙地卡羅模擬方法探討 European call option 的模擬價格與波動率 sigma 之間的關係。透過改變不同的波動率，可以觀察當標的資產價格不確定性提高時，買權價格會如何變化，進而了解波動率在選擇權定價中的重要性。

此外，本程式也比較兩種亂數使用方式對模擬結果的影響：一種是所有 sigma 共用同一批亂數路徑，另一種是在每一個 sigma 下重新產生亂數。這樣可以觀察同一組模擬條件下，曲線平滑與抖動的差異，並說明蒙地卡羅模擬結果除了受到金融參數影響，也會受到亂數抽樣方式影響。

2. 程式

這段主要是在做程式執行前的初始準備。首先載入數值運算、統計分配與繪圖所需的套件，讓後續可以進行亂數模擬、常態分配轉換、選擇權價格計算以及結果圖表繪製。接著設定圖表的字體大小，使輸出的圖表標題、座標軸與圖例更清楚易讀。最後先關閉先前可能存在的圖表視窗，避免舊圖影響本次模擬結果的呈現。整體來說，這一段是為後續的金融計算與視覺化分析建立基本環境。

```
1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 font_size = 14
6 plt.rcParams.update({"font.size": font_size})
7
8 plt.close("all")
```

下一步建立 Black-Scholes 選擇權定價函數，目的是用解析公式計算歐式選擇權的理論價格。函數中輸入的參數包含初始股價、履約價格、無風險利率、到期時間、波動率，以及選擇權型態。這些參數都是 Black-Scholes 模型中決定選擇權價格的重要因素。

在函數內，程式先計算 Black-Scholes 模型中的兩個中間變數。這兩個變數可以用來衡量在目前股價、履約價、利率、到期時間與波動率條件下，選擇權到期時落在價內的機率關係。由於 Black-Scholes 假設股價報酬服從常態分配，因此後續會透過標準常態累積機率來計算選擇權的理論價值。

接著，程式會依照選擇權型態判斷要計算買權或賣權。如果選擇權型態設定為買權，程式會計算 European call option 的理論價格；如果設定為賣權，則會計算 European put option 的理論價格。雖然本題主要分析的是 European call option，但保留賣權的寫法可以讓函數更完整，也與上課範例的寫法一致。

這個函數在本程式中的作用，是提供一個理論價格的基準。後續使用蒙地卡羅模擬得到的選擇權價格，可以和 Black-Scholes 理論價格做比較，幫助判斷模擬結果是否合理。不過本題的主要重點仍是觀察波動率改變時，European call option 的模擬價格如何變化。

```
11  #%%
12  def Black_Scholes(S0, K, r, T, sigma, option_type="call"):
13      d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
14      d2 = d1-sigma*np.sqrt(T)
15      if option_type == "call":
16          option_price = S0*stats.norm.cdf(d1)-K*np.exp(-r*T)*stats.norm.cdf(d2)
17      if option_type == "put":
18          option_price = K*np.exp(-r*T)*stats.norm.cdf(-d2)-S0*stats.norm.cdf(-d1)
19
20      return option_price
```

然後建立 European option 的蒙地卡羅模擬定價函數，目的是利用模擬出來的標準常態亂數，計算選擇權在到期日的股價、payoff，以及折現後的模擬價格。相較於前一段 Black-Scholes 函數是用解析公式計算理論價格，這一段則是用模擬方法估計選擇權價格。

首先，函數會接收初始股價、履約價格、無風險利率、到期時間、波動率、標準常態亂數，以及是否使用 moment matching 等設定。其中，標準常態亂數是蒙地卡羅模擬的核心，因為在 GBM 假設下，未來股價的變動會受到常態隨機變數影響。

若設定使用 moment matching，程式會先對亂數進行標準化處理，使每一組亂數樣本的平均數接近 0、標準差接近 1。這樣做的目的，是讓模擬使用的亂數更接近理論上的標準常態分配，減少有限樣本下可能產生的抽樣誤差，使模擬價格較穩定。

接著，程式會根據 GBM 的股價模型計算到期股價。這一步代表在風險中立測度下，利用目前股價、利率、波動率、到期時間與亂數，模擬出每一條路徑在

到期時的股價。當波動率越高時，模擬出的到期股價分布會越分散，這也是本題後續要觀察選擇權價格與波動率關係的基礎。

之後，程式會依照選擇權型態計算 payoff。如果是買權，payoff 取到期股價高於履約價的部分；如果是賣權，則取履約價高於到期股價的部分。本題主要分析 European call option，因此重點是買權 payoff。最後，程式將所有模擬路徑的 payoff 取平均，再用無風險利率折現回現在，得到蒙地卡羅模擬的選擇權價格。

整體而言，這個函數是本程式的核心計算步驟。它將 GBM 模型、亂數模擬、moment matching、payoff 計算與折現定價整合在一起，後續只要改變不同的波動率 sigma，就可以重複使用這個函數來觀察 European call option 模擬價格的變化。

```
22 def European_option_simulation(S0, K, r, T, sigma, Z, moment_matching=False, option_type="call"):
23     if moment_matching:
24         Z = (Z - np.mean(Z, axis=0)) / np.std(Z, axis=0)
25         # 每個 column 計算平均數和標準差，做 moment matching。
26
27     ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T + sigma * np.sqrt(T) * Z)
28
29     if option_type == "call":
30         payoff = np.maximum(ST - K, 0)
31     if option_type == "put":
32         payoff = np.maximum(K - ST, 0)
33
34     option_prices = np.exp(-r * T) * np.mean(payoff, axis=0)
35
36     return option_prices
```

接下來這一段是在設定本次蒙地卡羅模擬所需的基本參數與亂數產生方式。首先，程式設定初始股價與履約價格皆為 100，代表本題分析的是價平的 European call option。這樣的設定可以讓研究重點集中在波動率變化對買權價格的影響，而不是受到價內或價外程度太大的干擾。

無風險利率設定為 0.02，代表年利率為 2%；到期時間設定為 0.5，代表選擇權距離到期還有半年。這些參數會影響 GBM 模型下的到期股價模擬，也會影響 payoff 折現回現在的選擇權價格。選擇權型態設定為 call，表示本題主要計算 European call option；同時將 moment matching 設為 True，代表後續會對模擬亂數進行標準化校正，使亂數樣本更接近標準常態分配，以降低抽樣誤差。

接著，程式將模擬路徑數設定為 1000，表示每一次模擬會產生 1000 條可能的到期股價路徑。重複實驗次數設定為 1，表示每個波動率下只產生一組模擬價格。這樣的設定與上課內容中的基本蒙地卡羅模擬架構一致，可以在計算量不過大的情況下觀察選擇權價格變化。

亂數種子設定為 42，並在產生亂數前固定亂數種子。這樣做的目的是讓每次執行程式時都能產生相同的亂數序列，使模擬結果具有可重現性。由於本題後續

要比較不同波動率下的價格變化，固定亂數種子可以避免每次執行時圖形差異過大，方便觀察主要趨勢。

最後，程式設定亂數產生方式為 pseudo-random numbers。此寫法會先產生均勻分配亂數，再透過標準常態分配的反累積函數轉換成標準常態亂數，作為 GBM 模型中的隨機項。程式也保留其他產生標準常態亂數的方式，例如直接使用標準常態亂數函數或指定平均數與標準差的常態亂數函數，讓後續若要比較不同亂數產生方式時可以直接切換。

```
39  #%%
40  # 參數設定
41  S0 = 100.0
42  K = 100.0
43  r = 0.02
44  T = 0.5
45
46  option_type = "call"
47  moment_matching = True
48
49  m_paths = 1000
50  N = 1
51
52  seed = 42
53  np.random.seed(seed)
54
55  random_type = "pseudo"
56
57  if random_type == "pseudo":
58      u = np.random.rand(m_paths, N)
59      Z = stats.norm.ppf(u)
60
61  if random_type == "np.random.randn":
62      Z = np.random.randn(m_paths, N)
63
64  if random_type == "np.random.normal":
65      Z = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
```

下一步是本題的主要計算步驟，用來探討不同波動率對 European call option 模擬價格的影響。程式先建立一組波動率資料，範圍從 0.1 到 0.6，共取 26 個點，代表後續會分別在 26 種不同波動率下計算選擇權價格。

這裡使用等距取點的方式設定波動率。它的原理是把起始值 0.1 和結束值 0.6 之間的區間平均切成數個等距位置，並產生指定數量的數值。因為設定取 26 個點，所以程式會在 0.1 到 0.6 之間產生 26 個等距的波動率，包含起點 0.1，也包含終點 0.6。換句話說，波動率會從 10% 逐步增加到 60%，每一個相鄰波動率之間的距離相同。這樣設置的好處是可以讓圖表呈現連續而均勻的變化趨勢，方便觀察買權價格隨波動率上升時的變化。

選擇 0.1 到 0.6 作為波動率範圍，是為了涵蓋從低波動到較高波動的情況。0.1 代表標的資產價格波動相對較小，0.6 則代表股價波動較大。透過這個範圍，可以清楚比較不同市場不確定性下，European call option 模擬價格的變化。取 26

個點則可以讓曲線有足夠細緻的變化，不會因為取點太少而看不出趨勢，也不會因為取點過多而增加太多不必要的計算量。

接著，程式建立數個用來儲存結果的陣列，長度都與波動率資料數量相同。這些陣列分別用來保存每一個波動率下的 Black-Scholes 理論價格、蒙地卡羅模擬平均價格，以及後續另一種 for 迴圈寫法得到的模擬價格。先建立空陣列的好處是可以讓每一次迴圈計算出的結果依序存入對應位置，方便後面繪製圖表與比較。

在迴圈中，程式會依序取出每一個波動率，並在該波動率下計算 European call option 的價格。首先使用 Black-Scholes 函數計算理論價格，作為參考基準；接著使用蒙地卡羅模擬函數計算模擬價格。由於這一段使用的是先前已產生好的同一批標準常態亂數，因此不同波動率下使用的亂數來源一致，圖形會比較平滑，也比較適合觀察波動率本身對價格造成的影響。

最後，程式將每一個波動率下的理論價格與模擬平均價格存入對應的結果陣列中。完成迴圈後，就可以得到一整組「波動率與選擇權價格」的資料，作為後續繪圖的基礎。這一段的核心意義，是把單一選擇權價格計算擴展成一系列不同波動率下的價格比較，讓本題能夠從圖表中觀察 European call option 價格與波動率之間的關係。

```
68 #%%
69 # 探討不同波動率 sigma 對 European call option 模擬價格的影響
70 sigmas = np.linspace(0.1, 0.6, 26)
71
72 option_prices_mean = np.zeros(len(sigmas))
73 option_prices_for_loop_mean = np.zeros(len(sigmas))
74 option_prices_true = np.zeros(len(sigmas))
75
76 for i in range(len(sigmas)):
77     sigma = sigmas[i]
78
79     option_price_true = Black_Scholes(S0, K, r, T, sigma, option_type=option_type)
80     option_prices = European_option_simulation(S0, K, r, T, sigma, Z,
81                                               moment_matching=moment_matching,
82                                               option_type=option_type)
83
84     option_prices_true[i] = option_price_true
85     option_prices_mean[i] = np.mean(option_prices)
```

下一步用另一種 for 迴圈寫法重新進行模擬，目的是比較「同一批亂數重複使用」與「每次迴圈重新產生亂數」對模擬結果的影響。前一段程式是先產生一批標準常態亂數，然後在不同波動率下都使用同一批亂數來計算價格；而這一段則是在每一個波動率下，都重新產生一批新的亂數，再用新的亂數計算該波動率下的選擇權價格。

之所以要把 for 迴圈寫法分開，是因為這兩種寫法觀察的重點不同。第一種寫法讓所有波動率共用同一批亂數，因此不同波動率之間的差異主要來自波動率本身，曲線通常會比較平滑，適合用來觀察選擇權價格與波動率之間的理論趨勢。第二種寫法則是在每一個波動率下重新抽樣，因此每一個點除了受到波動率影響，也會受到該次亂數樣本不同所造成的抽樣誤差影響，曲線會比較容易出現抖動。

在這段程式中，程式先重新設定亂數種子，確保即使是在 for 迴圈內重新產生亂數，每次執行時仍然可以得到相同的亂數序列，使結果具有可重現性。接著，程式依序取出每一個波動率，並根據設定的亂數產生方式建立新的標準常態亂數。若使用 pseudo-random numbers，會先產生均勻分配亂數，再轉換成標準常態亂數；若使用其他設定，也可以直接產生標準常態亂數。

產生新的亂數後，程式會將該波動率與新產生的亂數放入 European option 的蒙地卡羅模擬函數中，計算該波動率下的選擇權模擬價格。由於每個波動率使用的亂數樣本不同，因此這一段得到的價格更能呈現蒙地卡羅模擬中抽樣誤差造成的自然波動。最後，程式將每個波動率下的模擬平均價格存入結果陣列，方便後續與前一種平滑寫法一起繪圖比較。

因此，分開寫兩種 for 迴圈的主要原因，是為了區分「參數變化造成的價格變化」與「亂數抽樣造成的模擬波動」。如果所有波動率共用同一批亂數，圖形會比較穩定；如果每個波動率重新抽亂數，圖形會比較抖動。兩者的定價邏輯相同，但呈現出的圖表平滑程度不同，這可以幫助說明蒙地卡羅模擬結果不只受到模型參數影響，也會受到亂數抽樣方式影響。

```
88  #%%
89  # 使用 for 迴圈重新產生亂數，比較不同 for 迴圈寫法之結果
90  np.random.seed(seed)
91
92  for i in range(len(sigmas)):
93      sigma = sigmas[i]
94
95      if random_type == "pseudo":
96          u_for_loop = np.random.rand(m_paths, N)
97          Z_for_loop = stats.norm.ppf(u_for_loop)
98
99      if random_type == "np.random.randn":
100         Z_for_loop = np.random.randn(m_paths, N)
101
102      if random_type == "np.random.normal":
103         Z_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
104
105         option_prices_for_loop = European_option_simulation(S0, K, r, T, sigma, Z_for_loop,
106                                                             moment_matching=moment_matching,
107                                                             option_type=option_type)
108
109         option_prices_for_loop_mean[i] = np.mean(option_prices_for_loop)
```

這一段是將前面計算出的結果繪製成圖表，用來比較兩種 for 迴圈寫法下，

European call option 模擬價格與波動率之間的關係。圖表大小設定為 10 乘 6，使畫面比例較清楚、方便閱讀。

圖中使用兩條藍色線表示不同模擬方式：藍色虛線代表所有波動率共用同一批亂數的結果，曲線較平滑；藍色實線代表每一個波動率都重新產生亂數的結果，曲線可能較容易出現抖動。透過這兩條線，可以直接觀察亂數產生位置不同對模擬結果穩定程度的影響。

最後，程式設定橫軸為波動率、縱軸為 European call option 價格，並加入標題、圖例與格線，讓圖表更容易判讀。完成版面調整後，程式會顯示圖表，作為後續結果分析的主要依據。

```
111 #%%
112 # 圖：比較兩種 for 迴圈寫法之結果
113 plt.figure(figsize=(10, 6))
114
115 plt.plot(sigmas,
116          option_prices_mean,
117          color="blue",
118          linewidth=2,
119          linestyle="--",
120          label="same Z")
121
122 plt.plot(sigmas,
123          option_prices_for_loop_mean,
124          color="blue",
125          linewidth=2,
126          linestyle="-",
127          label="Z in for loop")
128
129 plt.xlabel(r"$\sigma$")
130 plt.ylabel("European call option price")
131 plt.title("European call option simulation price and volatility")
132 plt.legend()
133 plt.grid(alpha=0.3)
134 plt.tight_layout()
135 plt.show()
```

3. 結果

由圖表可以看出，當波動率 σ 從 0.1 增加到 0.6 時，European call option 的模擬價格呈現明顯上升趨勢。當 σ 約為 0.1 時，買權價格大約在 3 到 4 之間；隨著 σ 提高，價格逐步上升，到 σ 接近 0.6 時，買權價格已經上升到約 17 左右。這表示在其他參數固定的情況下，波動率越高，European call option 的價值越高。

造成這個結果的原因，是買權的報酬具有不對稱性。對 European call option 而言，如果到期股價低於履約價格，買權可以選擇不履約，payoff 最低為 0；但如果到期股價高於履約價格，payoff 會隨著股價上升而增加。因此，當波動率提高時，股價未來可能出現更大的上下波動。雖然股價下跌的可能性也增加，但

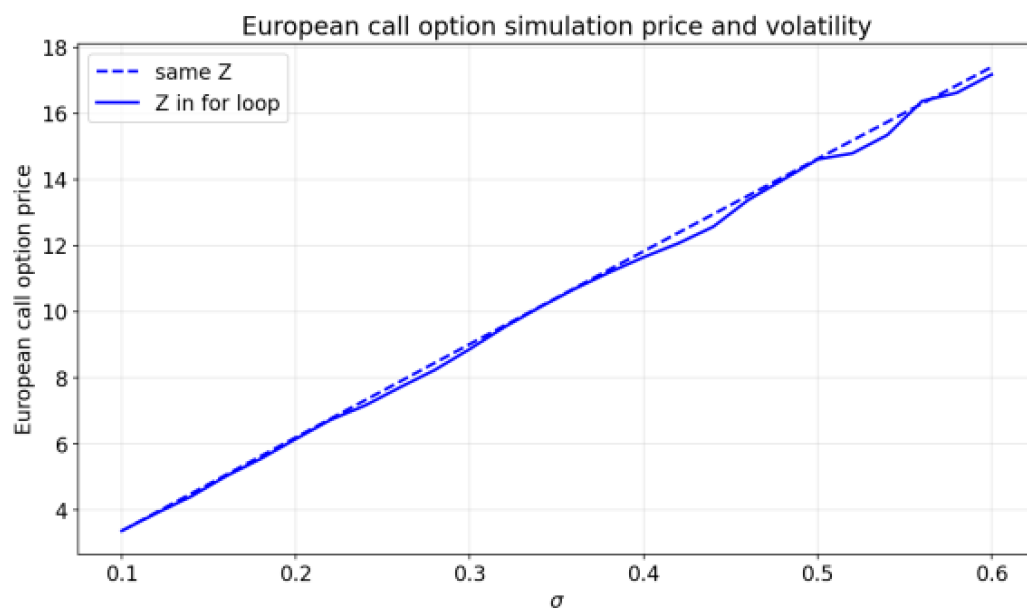
買權的下方損失被限制在 0；相反地，股價大幅上漲時會帶來更高的 payoff，所以整體期望 payoff 會上升，折現後的選擇權價格也會上升。

圖中有兩條藍色線，分別代表兩種不同的 for 迴圈寫法。藍色虛線 same Z 表示所有波動率都使用同一批亂數來模擬，因此不同 sigma 之間的差異主要來自波動率本身，曲線看起來相對平滑。藍色實線 Z in for loop 則表示每一個波動率下都重新產生亂數，因此每個點都受到不同亂數樣本的影響，會出現較明顯的上下波動。

從圖表中可以看到，兩條線的整體方向幾乎一致，皆隨著 sigma 上升而上升。這代表不論採用哪一種亂數使用方式，European call option 價格與波動率之間的正向關係都很明確。也就是說，波動率越高、買權價格越高這個結論並不是由特定亂數樣本造成，而是來自買權本身的 payoff 結構與 GBM 模型下股價分布的特性。

不過，兩條線在局部位置仍有些微差異。例如在較高波動率區間，實線相對於虛線有時略高、有時略低，這是因為每一個波動率下重新抽樣會帶入蒙地卡羅模擬的抽樣誤差。由於本程式設定的模擬路徑數為 1000，樣本數雖然足以呈現主要趨勢，但仍不是無限大，因此模擬價格會受到亂數樣本影響而產生小幅抖動。

整體而言，這張圖同時說明了兩件事。第一，European call option 的模擬價格會隨著波動率 sigma 增加而上升，顯示波動率是影響買權價格的重要因素。第二，for 迴圈中亂數產生的位置會影響圖表的平滑程度：共用同一批亂數可以更清楚呈現參數變化造成的趨勢，而每次重新產生亂數則能呈現蒙地卡羅模擬中的抽樣波動。雖然兩種寫法的曲線細節不同，但共同支持本題的主要結論。



4. 證據

本程式的目的，是探討在 GBM 假設下，European call option 的模擬價格與波動率 σ 之間的關係，並進一步比較不同 for 迴圈亂數寫法對模擬結果的影響。為了達成這個目的，程式先建立 Black-Scholes 理論價格函數與蒙地卡羅模擬函數，再設定初始股價、履約價格、無風險利率、到期時間、模擬路徑數、亂數種子與 moment matching 等參數。

在步驟上，程式先將波動率設定為從 0.1 到 0.6 的等距數列，代表由低波動到高波動的不同市場狀態。接著分別使用兩種方式進行模擬：第一種是所有波動率共用同一批標準常態亂數，用來觀察較平滑的價格變化趨勢；第二種是在每一個波動率下重新產生亂數，用來呈現蒙地卡羅抽樣造成的自然波動。最後，程式將兩種模擬結果繪製在同一張圖上，方便直接比較。

從圖表可以看到，無論使用同一批亂數或在 for 迴圈中重新產生亂數，European call option 的模擬價格都會隨著波動率上升而增加。這正好回應本題目的：波動率提高會使到期股價分布更分散，增加買權獲得較高 payoff 的可能性，因此買權價格上升。同時，圖中的虛線較平滑、實線較有抖動，也證明不同 for 迴圈亂數寫法會影響模擬曲線的穩定程度，但不會改變主要結論。

5. 結論

綜合本題結果可以得知，在假設股價服從 GBM 且其他參數固定的情況下，European call option 的模擬價格會隨著波動率 σ 增加而上升。這表示波動率是影響買權價格的重要因素，當市場不確定性越高時，標的股價大幅上漲的可能性也提高，因此買權的期望 payoff 增加，折現後的選擇權價格也會提高。

此外，兩種 for 迴圈寫法雖然會造成圖表平滑程度不同，但不會改變整體趨勢。共用同一批亂數時，曲線較平滑，較適合觀察波動率對價格的主要影響；在每一個波動率下重新產生亂數時，曲線較容易抖動，能呈現蒙地卡羅模擬受到抽樣誤差影響的特性。因此，本題不僅驗證了買權價格與波動率之間的正向關係，也說明亂數產生方式會影響模擬結果的呈現方式。

（二）題目：

（a）增加比較不同波動（ σ ）所設定的亂數種子（random seed）固定與不固定比較，並由此得到結論。

1. 目的

本題延續子主題 A 的設定，目的在於進一步比較在不同波動率 σ 下，固定亂數種子與不固定亂數種子對 European call option 蒙地卡羅模擬價格的影響。前一題主要觀察波動率與買權價格之間的關係，而本題則加入亂數種子是否固定的比較，分析模擬結果是否會因為使用不同亂數樣本而產生差異。

透過固定亂數種子，可以讓每次執行程式時產生相同的亂數序列，使模擬結果具有可重現性；而不固定亂數種子則會讓每次模擬使用不同的亂數樣本，更能呈現蒙地卡羅模擬本身的隨機性與抽樣波動。因此，本題的目的不只是觀察 European call option 價格是否仍會隨著波動率上升而增加，也要比較固定與不固定亂數種子下，模擬曲線在穩定程度與數值表現上的差異。

2. 程式

一開始的步驟是延續上一題的程式架構，因此前面的設定可以簡單說明。程式一開始先載入數值運算、統計分配與繪圖所需的套件，並設定圖表字體大小與清除舊圖表，確保後續模擬與圖形輸出能正常進行。接著沿用 Black-Scholes 理論價格函數與 European option 蒙地卡羅模擬函數，作為計算選擇權價格的基礎。

在參數設定方面，本題維持與上一題相同的主要條件，包括初始股價為 100、履約價格為 100、無風險利率為 0.02、到期時間為 0.5、選擇權型態為買權、模擬路徑數為 1000，並且使用 moment matching。這樣設定的目的，是讓其他金融參數保持固定，使本題能專注比較「固定亂數種子」與「不固定亂數種子」對模擬結果的影響。

接著，程式同樣將波動率設定為從 0.1 到 0.6，共 26 個等距點，用來觀察不同波動率下的 European call option 模擬價格變化。與上一題不同的是，本題先建立兩組儲存結果的陣列，分別用來保存固定亂數種子與不固定亂數種子時的模擬平均價格。後續再用相同的模型、相同的波動率範圍與相同的參數設定，單純比較亂數種子是否固定所造成的差異。

```
1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 font_size = 14
6 plt.rcParams.update({"font.size": font_size})
7
8 plt.close("all")
```

```

11  #%%
12  def Black_Scholes(S0, K, r, T, sigma, option_type="call"):
13      d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
14      d2 = d1-sigma*np.sqrt(T)
15      if option_type == "call":
16          option_price = S0*stats.norm.cdf(d1)-K*np.exp(-r*T)*stats.norm.cdf(d2)
17      if option_type == "put":
18          option_price = K*np.exp(-r*T)*stats.norm.cdf(-d2)-S0*stats.norm.cdf(-d1)
19
20      return option_price
21
22  def European_option_simulation(S0, K, r, T, sigma, Z, moment_matching=False, option_type="call"):
23      if moment_matching:
24          Z = (Z-np.mean(Z, axis=0))/np.std(Z, axis=0)
25          # 每個 column 計算平均數和標準差，做 moment matching。
26
27      ST = S0*np.exp((r-0.5*sigma**2)*T+sigma*np.sqrt(T)*Z)
28
29      if option_type == "call":
30          payoff = np.maximum(ST-K, 0)
31      if option_type == "put":
32          payoff = np.maximum(K-ST, 0)
33
34      option_prices = np.exp(-r*T)*np.mean(payoff, axis=0)
35
36      return option_prices

```

```

39  #%%
40  # 參數設定
41  S0 = 100.0
42  K = 100.0
43  r = 0.02
44  T = 0.5
45
46  option_type = "call"
47  moment_matching = True
48
49  m_paths = 1000
50  N = 1
51
52  random_type = "pseudo"
53
54
55  #%%
56  # 探討不同波動率 sigma 對 European call option 模擬價格的影響
57  sigmas = np.linspace(0.1, 0.6, 26)
58
59  option_prices_seed_for_loop_mean = np.zeros(len(sigmas))
60  option_prices_no_seed_for_loop_mean = np.zeros(len(sigmas))
61

```

下一步是在計算「固定亂數種子」情況下，不同波動率對 European call option 模擬價格的影響。程式先將亂數種子固定為 42，而且是將亂數種子放在 for 迴圈外面，讓每次執行程式時都會從同一組亂數序列開始產生亂數。因此，即使後續在每一個波動率下重新產生亂數，整體結果仍然具有可重現性。

本題特別採用「抖動較明顯」的 for 迴圈寫法，也就是將亂數產生的步驟放在 for 迴圈內，使每一個 sigma 都重新產生一批亂數，再用這批亂數計算該波動率下的選擇權價格。這樣寫的目的，是讓圖表更明顯呈現蒙地卡羅模擬中亂數抽樣造成的影響。若所有波動率都共用同一批亂數，曲線會比較平滑，但比較不容易看出不同亂數樣本對模擬價格造成的波動；而每一個波動率重新抽樣，則能讓抽樣誤差更清楚地反映在圖表上。

需要注意的是，抖動明顯的寫法並不是把亂數種子放進 for 迴圈內，而是把亂數

種子固定在 for 迴圈外，並在 for 迴圈內重新產生亂數。如果把亂數種子放在 for 迴圈內，每一次迴圈都會從同一個亂數序列重新開始，反而會使每個 sigma 使用到過於相同的亂數樣本，無法呈現真正的重新抽樣效果。因此，本題的寫法可以同時保留可重現性與抽樣波動。

在迴圈中，程式會依序取出每一個波動率，並根據設定的亂數產生方式建立新的標準常態亂數。本程式使用 pseudo-random numbers，因此會先產生均勻分配亂數，再轉換成標準常態亂數，作為 GBM 模型中的隨機項。接著，程式將該波動率與新產生的亂數放入蒙地卡羅模擬函數中，計算該波動率下的 European call option 價格。

由於這一段是固定亂數種子，因此雖然每個 sigma 都重新抽亂數，但每次重新執行程式時，產生的整體亂數序列仍會相同，圖表結果也會一致。這樣的設計可以同時達到兩個目的：一方面保留抖動效果，讓抽樣誤差更明顯；另一方面又讓結果可以重複產生，方便報告分析與檢查。最後，程式將每個波動率下的模擬平均價格存入結果陣列，作為後續與不固定亂數種子結果比較的基礎。

```
63  #%%
64  # 固定亂數種子，並在 for 迴圈中每一個 sigma 重新產生亂數
65  seed = 42
66  np.random.seed(seed)
67
68  for i in range(len(sigmals)):
69      sigma = sigmals[i]
70
71      if random_type == "pseudo":
72          u_seed_for_loop = np.random.rand(m_paths, N)
73          Z_seed_for_loop = stats.norm.ppf(u_seed_for_loop)
74
75      if random_type == "np.random.randn":
76          Z_seed_for_loop = np.random.randn(m_paths, N)
77
78      if random_type == "np.random.normal":
79          Z_seed_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
80
81      option_prices_seed_for_loop = European_option_simulation(S0, K, r, T, sigma, Z_seed_for_loop,
82                                                                moment_matching=moment_matching,
83                                                                option_type=option_type)
84
85      option_prices_seed_for_loop_mean[i] = np.mean(option_prices_seed_for_loop)
```

然後這一段是在計算「不固定亂數種子」情況下，不同波動率對 European call option 模擬價格的影響。程式先將亂數種子設定為不固定，表示每次執行程式時，亂數會由系統重新決定起始狀態，因此每次產生的亂數樣本都可能不同。這樣的設定可以用來觀察蒙地卡羅模擬在不同亂數樣本下可能出現的自然變動。

這一段同樣採用抖動較明顯的 for 迴圈寫法，也就是在每一個 sigma 下都重新產生一批亂數，再用這批亂數計算該波動率下的選擇權價格。這樣做的目的，是讓每一個波動率都對應到不同的亂數樣本，因此圖表會更明顯呈現抽樣誤差造

成的波動。與固定亂數種子的情況相比，不固定亂數種子不只是在每個 σ 下重新抽樣，連每次重新執行整個程式時，抽到的亂數序列也可能改變。

在迴圈中，程式會依序取出每一個波動率，並根據設定的亂數產生方式建立新的標準常態亂數。本程式使用 pseudo-random numbers，因此會先產生均勻分配亂數，再轉換成標準常態亂數，作為 GBM 模型中的隨機項。接著，程式將該波動率與新產生的亂數放入 European option 的蒙地卡羅模擬函數中，計算該波動率下的模擬價格。

最後，程式將每一個波動率下的模擬平均價格存入不固定亂數種子的結果陣列中。這組結果會在後續與固定亂數種子的結果一起繪圖比較。透過這樣的設計，可以觀察固定 seed 與不固定 seed 在相同模型與相同參數下，是否會因為亂數樣本不同而造成曲線細節上的差異。整體而言，這一段的重點是呈現不固定亂數種子下的蒙地卡羅抽樣波動，並作為固定亂數種子結果的比較對象。

```
88 #%%
89 # 不固定亂數種子，並在 for 迴圈中每一個 sigma 重新產生亂數
90 np.random.seed(None)
91
92 for i in range(len(sigmas)):
93     sigma = sigmas[i]
94
95     if random_type == "pseudo":
96         u_no_seed_for_loop = np.random.rand(m_paths, N)
97         Z_no_seed_for_loop = stats.norm.ppf(u_no_seed_for_loop)
98
99     if random_type == "np.random.randn":
100         Z_no_seed_for_loop = np.random.randn(m_paths, N)
101
102     if random_type == "np.random.normal":
103         Z_no_seed_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
104
105     option_prices_no_seed_for_loop = European_option_simulation(S0, K, r, T, sigma, Z_no_seed_for_loop,
106                                                                moment_matching=moment_matching,
107                                                                option_type=option_type)
108
109     option_prices_no_seed_for_loop_mean[i] = np.mean(option_prices_no_seed_for_loop)
```

這一段是將前面計算出的固定亂數種子與不固定亂數種子的模擬結果繪製成圖表。圖表大小設定為 10 乘 6，讓曲線與標籤能清楚呈現。

圖中以藍色實線表示固定亂數種子 seed = 42 的模擬價格，以紅色實線表示不固定亂數種子的模擬價格。橫軸為波動率 σ ，縱軸為 European call option 的模擬價格，用來比較兩種亂數種子設定下，選擇權價格隨波動率變化的情形。

最後，程式加入標題、圖例與格線，並調整版面後顯示圖表，作為後續結果分析的依據。

```
112 #%%
113 # 圖：比較固定與不固定亂數種子下，European call option 模擬價格與 sigma 之關係
114 plt.figure(figsize=(10, 6))
```

```

116 plt.plot(sigas,
117           option_prices_seed_for_loop_mean,
118           color="blue",
119           linewidth=2,
120           linestyle="-",
121           label="fixed seed = 42")
122
123 plt.plot(sigas,
124           option_prices_no_seed_for_loop_mean,
125           color="red",
126           linewidth=2,
127           linestyle="-",
128           label="unfixed seed")
129
130 plt.xlabel(r"$\sigma$")
131 plt.ylabel("European call option price")
132 plt.title("European call option simulation price: fixed seed and unfixed seed")
133 plt.legend()
134 plt.grid(alpha=0.3)
135 plt.tight_layout()
136 plt.show()

```

3. 結果

由圖表可以看出，不論是固定亂數種子或不固定亂數種子，European call option 的模擬價格都隨著波動率 σ 上升而增加。當 σ 約為 0.1 時，兩條線的價格大約都在 3 到 4 之間；當 σ 增加到 0.6 附近時，價格上升到約 17 左右。這表示在其他參數固定的情況下，波動率越高，European call option 的模擬價格越高，與前一題得到的結論一致。

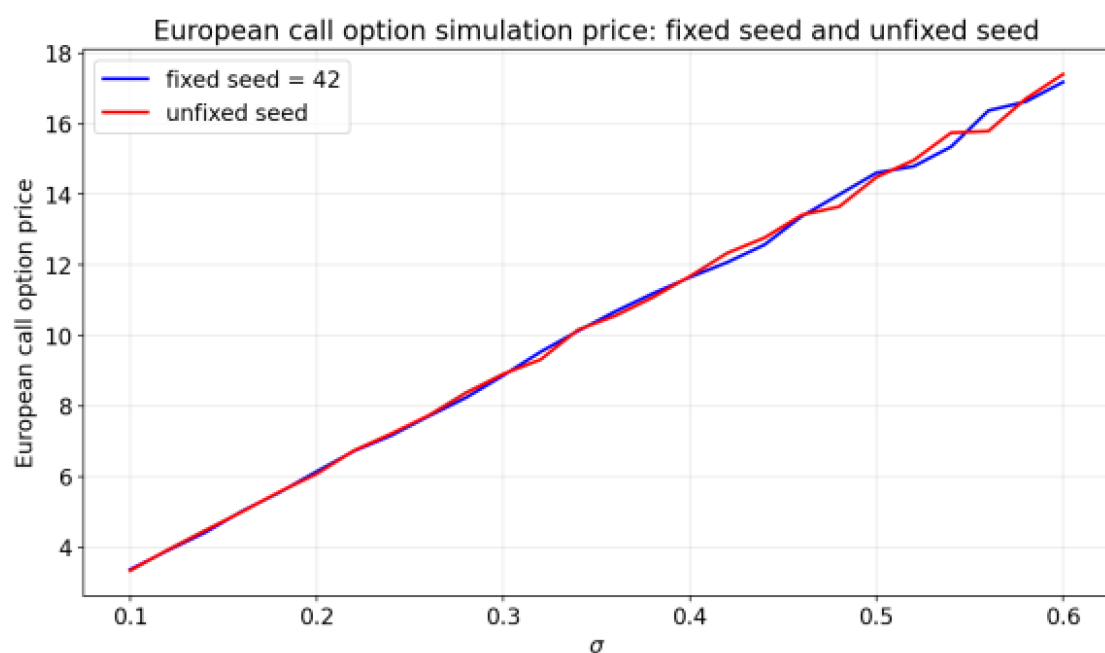
造成這個上升趨勢的原因，是買權 payoff 本身具有不對稱性。當到期股價低於履約價格時，買權可以選擇不履約，因此 payoff 最低為 0；但當到期股價高於履約價格時，payoff 會隨股價上升而增加。因此，波動率提高會讓到期股價分布更加分散，雖然股價下跌的可能性也提高，但買權的下方損失有限；相反地，股價大幅上漲時可以帶來更高 payoff，所以買權價格會隨著波動率增加而上升。

圖中藍色線代表固定亂數種子 $\text{seed} = 42$ ，紅色線代表不固定亂數種子。兩條線整體非常接近，表示固定與不固定亂數種子並不會改變 European call option 價格與波動率之間的主要關係。換句話說，波動率越高、買權價格越高這個結果，主要來自模型與買權 payoff 結構，而不是特定亂數種子造成的偶然結果。

不過，兩條線在局部位置仍有些微差異，尤其在中高波動率區間可以看到兩條線有時會略微分開。這是因為固定亂數種子與不固定亂數種子使用的亂數樣本不同，而本題又採用在每一個 σ 下重新產生亂數的 for 迴圈寫法，因此每一個波動率的模擬價格都會受到當次抽樣結果影響。由於模擬路徑數為 1000，樣本數有限，所以抽樣誤差會在曲線上呈現小幅抖動。

固定亂數種子的優點，是每次執行程式都會得到相同的亂數序列，因此圖表可以重複產生，方便檢查與比較。不固定亂數種子則更能反映蒙地卡羅模擬本身的隨機性，每次執行時可能得到略有不同的曲線。從圖中可以看出，即使亂數樣本不同，兩條線仍然貼近且趨勢一致，代表本題的模擬結果具有穩定的方向性。

整體而言，這張圖說明固定 seed 與不固定 seed 主要影響的是模擬曲線的細節與可重現性，而不是改變選擇權價格與波動率的基本關係。固定 seed 讓結果可重現，不固定 seed 則呈現抽樣波動；但兩者都支持同一個結論：在 GBM 假設下，European call option 的模擬價格會隨著波動率 σ 增加而上升。



4. 證據

本題是為了比較固定亂數種子與不固定亂數種子時，European call option 模擬價格是否會出現差異。程式延續前一題的參數設定，並在每一個波動率下重新產生亂數，分別計算固定 seed=42 與不固定 seed 的模擬價格，再將兩組結果畫在同一張圖中比較。

從圖表可以看到，藍色線代表固定 seed，紅色線代表不固定 seed，兩條線都隨著波動率 σ 增加而上升。這表示不論亂數種子是否固定，European call option 價格與波動率之間仍呈現正向關係。也就是說，波動率越高、買權價格越高的結果並不是特定 seed 造成的，而是在不同亂數樣本下都能觀察到。

同時，兩條線在局部位置有些微差距，這就是亂數樣本不同所造成的抽樣誤差。固定 seed 的結果可以重複產生，不固定 seed 則反映蒙地卡羅模擬的隨機波

動。整體而言，圖表證明亂數種子設定主要影響曲線細節與可重現性，但不會改變選擇權價格隨波動率上升的主要趨勢。

5. 結論

綜合本題結果可以得知，在相同模型與參數設定下，固定亂數種子與不固定亂數種子都會得到相同的主要趨勢：European call option 的模擬價格會隨著波動率 σ 增加而上升。這表示買權價格與波動率之間的正向關係並不是由特定亂數種子造成，而是來自 GBM 模型與買權 payoff 結構本身。

固定亂數種子的優點是讓模擬結果具有可重現性，適合用於報告分析、程式檢查與不同參數的比較；不固定亂數種子則能呈現蒙地卡羅模擬在不同亂數樣本下可能產生的自然波動。雖然兩者在局部數值上會有些微差異，但不會改變整體結論。因此，本題說明亂數種子設定主要影響模擬結果的穩定性與可重現性，而不會改變 European call option 價格隨波動率上升的基本關係。

（三）題目：

（b）增加比較使用 moment matching 與不使用 moment matching 之結果比較，並由此得到結論。

1. 目的

本題延續前一題固定亂數種子與不固定亂數種子的比較，進一步加入是否使用 moment matching 的設定，目的是觀察在不同亂數種子條件下，使用與不使用 moment matching 對 European call option 蒙地卡羅模擬價格的影響。也就是說，本題同時比較固定亂數種子、不固定亂數種子、使用 moment matching，以及不使用 moment matching 這幾種情況下，模擬結果是否會出現差異。

Moment matching 的目的，是將模擬中產生的標準常態亂數重新校正，使樣本平均數接近 0、標準差接近 1，讓有限樣本下的亂數更接近理論標準常態分配。透過這個比較，可以觀察 moment matching 是否能降低抽樣誤差，使模擬曲線更穩定，也可以檢查即使在不同亂數種子與不同 moment matching 設定下，European call option 價格是否仍然會隨著波動率 σ 上升而增加。

2. 程式

這一題也是延續前一題的程式架構，所以相同的部分可以簡單說明。程式一開始同樣載入數值運算、統計分配與繪圖所需的套件，並設定圖表字體大小與清除舊圖表，作為後續模擬與繪圖的準備。接著也沿用 Black-Scholes 理論價格函數與 European option 蒙地卡羅模擬函數，用來計算選擇權價格。

在參數設定方面，本題同樣維持初始股價為 100、履約價格為 100、無風險利率為 0.02、到期時間為 0.5、選擇權型態為買權、模擬路徑數為 1000、重複實驗次數為 1，並使用 pseudo-random numbers。這些設定都與前一題一致，是為了讓本題可以專注比較「是否使用 moment matching」造成的影響。

本題的主要差異，是在前一題固定亂數種子與不固定亂數種子的架構上，再加入 moment matching 的 True 與 False 比較。因此程式先設定波動率從 0.1 到 0.6，共 26 個等距點，並建立四組結果陣列，分別儲存固定亂數種子且使用 moment matching、不固定亂數種子且使用 moment matching、固定亂數種子但不使用 moment matching，以及不固定亂數種子但不使用 moment matching 的模擬平均價格。透過這四組結果，後續可以直接比較亂數種子與 moment matching 對模擬價格的影響。

```
1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 font_size = 14
6 plt.rcParams.update({"font.size": font_size})
7
8 plt.close("all")
```

```
11 #%%
12 def Black_Scholes(S0, K, r, T, sigma, option_type="call"):
13     d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
14     d2 = d1-sigma*np.sqrt(T)
15     if option_type == "call":
16         option_price = S0*stats.norm.cdf(d1)-K*np.exp(-r*T)*stats.norm.cdf(d2)
17     if option_type == "put":
18         option_price = K*np.exp(-r*T)*stats.norm.cdf(-d2)-S0*stats.norm.cdf(-d1)
19
20     return option_price
21
22 def European_option_simulation(S0, K, r, T, sigma, Z, moment_matching=False, option_type="call"):
23     if moment_matching:
24         Z = (Z-np.mean(Z, axis=0))/np.std(Z, axis=0)
25         # 每個 column 計算平均數和標準差，做 moment matching。
26
27     ST = S0*np.exp((r-0.5*sigma**2)*T+sigma*np.sqrt(T)*Z)
28
29     if option_type == "call":
30         payoff = np.maximum(ST-K, 0)
31     if option_type == "put":
32         payoff = np.maximum(K-ST, 0)
33
34     option_prices = np.exp(-r*T)*np.mean(payoff, axis=0)
35
36     return option_prices
```



```

39  #%%
40  # 參數設定
41  S0 = 100.0
42  K = 100.0
43  r = 0.02
44  T = 0.5
45
46  option_type = "call"
47
48  m_paths = 1000
49  N = 1
50
51  random_type = "pseudo"
52
53
54  #%%
55  # 探討不同波動率 sigma 對 European call option 模擬價格的影響
56  sigmas = np.linspace(0.1, 0.6, 26)
57
58  option_prices_seed_MM_mean = np.zeros(len(sigmas))
59  option_prices_no_seed_MM_mean = np.zeros(len(sigmas))
60  option_prices_seed_no_MM_mean = np.zeros(len(sigmas))
61  option_prices_no_seed_no_MM_mean = np.zeros(len(sigmas))

```

下一步是在固定亂數種子的情況下，同時比較使用與不使用 moment matching 時，European call option 的模擬價格。程式先將亂數種子固定為 42，並且把亂數種子設定在 for 迴圈外，這樣每次執行程式時都會從相同的亂數序列開始，使模擬結果具有可重現性。

接著，程式採用較容易呈現抖動的 for 迴圈寫法，也就是在每一個波動率 sigma 下都重新產生一批亂數。這樣每個 sigma 使用的亂數樣本都不同，因此圖表可以呈現蒙地卡羅抽樣誤差造成的波動。不過因為 seed 固定在迴圈外，所以即使每個 sigma 都重新抽樣，每次重新執行程式時仍會得到相同結果。

在每一次迴圈中，程式會先依照設定的亂數產生方式建立標準常態亂數。本題使用 pseudo-random numbers，因此會先產生均勻分配亂數，再轉換成標準常態亂數。接著，同一批亂數會被拿來計算兩種情況：第一種是使用 moment matching，第二種是不使用 moment matching。這樣做的好處是兩者使用的亂數來源相同，因此可以更公平地比較 moment matching 本身對模擬價格的影響。

使用 moment matching 時，亂數會被重新校正，使樣本平均數接近 0、標準差接近 1；不使用 moment matching 時，則保留原始亂數樣本。最後，程式分別計算兩種情況下的 European call option 模擬價格，並將平均價格存入對應的結果陣列中。這一段的重點，是在固定亂數種子且每個波動率重新抽樣的條件下，比較 moment matching 是否會影響模擬價格的穩定性與數值表現。


```

64  #%%
65  # 固定亂數種子，並在 for 迴圈中每一個 sigma 重新產生亂數
66  seed = 42
67  np.random.seed(seed)
68
69  for i in range(len(sigmals)):
70      sigma = sigmas[i]
71
72      if random_type == "pseudo":
73          u_seed_for_loop = np.random.rand(m_paths, N)
74          Z_seed_for_loop = stats.norm.ppf(u_seed_for_loop)
75
76      if random_type == "np.random.randn":
77          Z_seed_for_loop = np.random.randn(m_paths, N)
78
79      if random_type == "np.random.normal":
80          Z_seed_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
81
82      option_prices_seed_MM = European_option_simulation(S0, K, r, T, sigma, Z_seed_for_loop,
83                                                         moment_matching=True,
84                                                         option_type=option_type)
85
86      option_prices_seed_no_MM = European_option_simulation(S0, K, r, T, sigma, Z_seed_for_loop,
87                                                            moment_matching=False,
88                                                            option_type=option_type)
89
90      option_prices_seed_MM_mean[i] = np.mean(option_prices_seed_MM)
91      option_prices_seed_no_MM_mean[i] = np.mean(option_prices_seed_no_MM)

```

然後這一段是在不固定亂數種子的情況下，同時比較使用與不使用 moment matching 時，European call option 的模擬價格。程式先將亂數種子設定為不固定，代表每次執行程式時，亂數序列都可能不同，因此可以觀察蒙地卡羅模擬在不同亂數樣本下的自然變動。

這裡同樣採用抖動較明顯的 for 迴圈寫法，也就是在每一個波動率 sigma 下都重新產生一批亂數。這樣每一個 sigma 都會使用不同的亂數樣本，因此圖表會更明顯呈現抽樣誤差造成的波動。與固定亂數種子的情況不同，不固定亂數種子除了每個 sigma 會重新抽樣之外，每次重新執行整個程式時也可能產生不同的結果。

在每一次迴圈中，程式會先依照設定的亂數產生方式建立標準常態亂數。本題使用 pseudo-random numbers，因此會先產生均勻分配亂數，再轉換成標準常態亂數。接著，程式使用同一批亂數分別計算兩種情況：一種是使用 moment matching，另一種是不使用 moment matching。這樣可以在相同亂數來源下比較 moment matching 的影響，避免兩種結果差異完全來自亂數樣本不同。

使用 moment matching 時，程式會將亂數樣本重新標準化，使其平均數接近 0、標準差接近 1，藉此降低有限樣本造成的抽樣偏差；不使用 moment matching 時，則直接保留原始亂數樣本。因此，兩者的差異可以用來觀察 moment matching 是否能讓模擬價格更穩定。最後，程式將使用與不使用 moment matching 的模擬平均價格分別存入對應的結果陣列，作為後續圖表比較的資料來源。

```

94  #%%
95  # 不固定亂數種子，並在 for 迴圈中每一個 sigma 重新產生亂數
96  np.random.seed(None)
97
98  for i in range(len(sigmals)):
99      sigma = sigmals[i]
100
101      if random_type == "pseudo":
102          u_no_seed_for_loop = np.random.rand(m_paths, N)
103          Z_no_seed_for_loop = stats.norm.ppf(u_no_seed_for_loop)
104
105      if random_type == "np.random.randn":
106          Z_no_seed_for_loop = np.random.randn(m_paths, N)
107
108      if random_type == "np.random.normal":
109          Z_no_seed_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
110
111      option_prices_no_seed_MM = European_option_simulation(S0, K, r, T, sigma, Z_no_seed_for_loop,
112                                                            moment_matching=True,
113                                                            option_type=option_type)
114
115      option_prices_no_seed_no_MM = European_option_simulation(S0, K, r, T, sigma, Z_no_seed_for_loop,
116                                                                moment_matching=False,
117                                                                option_type=option_type)
118
119      option_prices_no_seed_MM_mean[i] = np.mean(option_prices_no_seed_MM)
120      option_prices_no_seed_no_MM_mean[i] = np.mean(option_prices_no_seed_no_MM)

```

這一段是將前面四種模擬結果繪製在同一張圖中，方便比較不同亂數種子設定與是否使用 moment matching 對 European call option 模擬價格的影響。圖表大小設定為 10 乘 6，使各條曲線與圖例能清楚呈現。

圖中分別用四種顏色代表四種情況：藍色線代表固定亂數種子且使用 moment matching，紅色線代表不固定亂數種子且使用 moment matching，綠色線代表固定亂數種子但不使用 moment matching，黃色線代表不固定亂數種子且不使用 moment matching。橫軸為波動率 sigma，縱軸為 European call option 的模擬價格。

最後，程式加入圖表標題、座標軸標籤、圖例與格線，並調整版面後顯示圖表，作為後續分析四種設定下模擬結果差異的依據。

```

123  #%%
124  # 圖：比較固定與不固定亂數種子下，有無使用 moment matching 之模擬結果
125  plt.figure(figsize=(10, 6))
126
127  plt.plot(sigmals,
128           option_prices_seed_MM_mean,
129           color="blue",
130           linewidth=2,
131           label="fixed seed, MM")
132
133  plt.plot(sigmals,
134           option_prices_no_seed_MM_mean,
135           color="red",
136           linewidth=2,
137           label="unfixed seed, MM")
138
139  plt.plot(sigmals,
140           option_prices_seed_no_MM_mean,
141           color="green",
142           linewidth=2,
143           label="fixed seed, no MM")

```

```

145 plt.plot(sigmas,
146           option_prices_no_seed_no_MM_mean,
147           color="gold",
148           linewidth=2,
149           label="unfixed seed, no MM")
150
151 plt.xlabel(r"$\sigma$")
152 plt.ylabel("European call option price")
153 plt.title("European call option simulation price: moment matching comparison")
154 plt.legend()
155 plt.grid(alpha=0.3)
156 plt.tight_layout()
157 plt.show()

```

3. 結果

由圖表可以看出，四條曲線整體都隨著波動率 σ 上升而增加。當 σ 約為 0.1 時，European call option 的模擬價格大約落在 3 到 4 之間；當 σ 上升到 0.6 附近時，價格大約上升到 16 到 18 左右。這表示不論亂數種子是否固定，也不論是否使用 moment matching，買權價格與波動率之間都呈現正向關係。

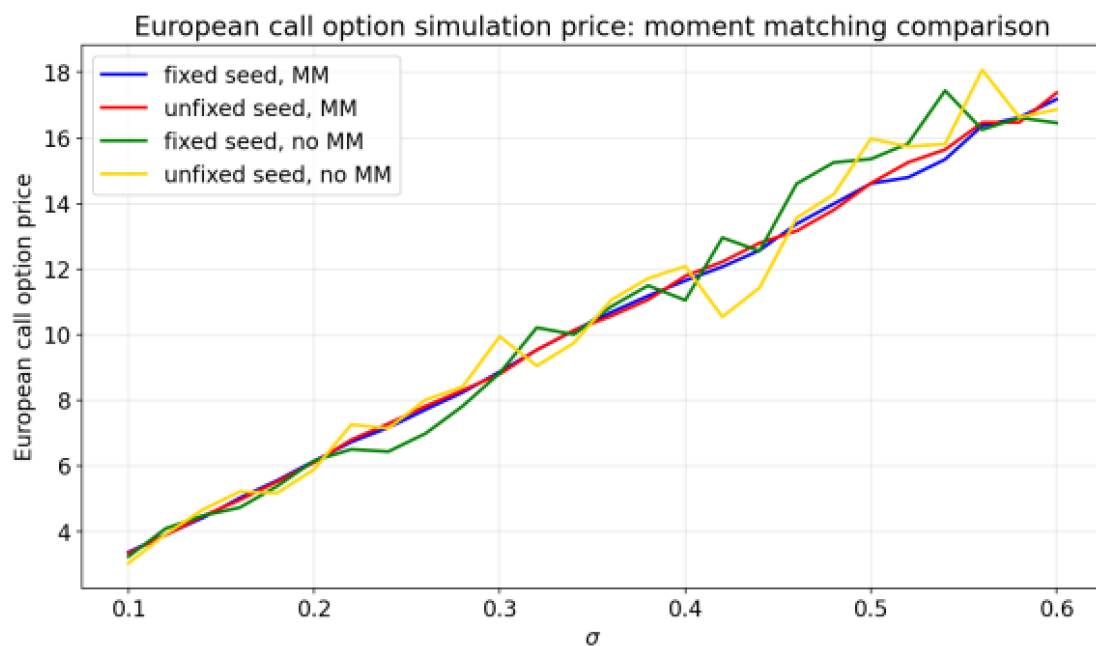
造成這個結果的主要原因，是 European call option 的 payoff 結構具有不對稱性。當到期股價低於履約價格時，買權可以不履約，payoff 最低為 0；但當到期股價高於履約價格時，payoff 會隨著股價上升而增加。因此，波動率越高，股價未來分布越分散，雖然下跌風險也提高，但買權的下方損失有限，而上方獲利空間較大，所以整體期望 payoff 會增加，模擬價格也會上升。

圖中的藍色線與紅色線代表使用 moment matching 的結果，其中藍色線為固定亂數種子，紅色線為不固定亂數種子。可以看到，這兩條線相對平穩，彼此也非常接近，表示使用 moment matching 後，不同亂數種子設定下的模擬價格差異較小。這是因為 moment matching 會將亂數樣本重新校正，使樣本平均數接近 0、標準差接近 1，因此能降低部分由亂數樣本偏差造成的抽樣誤差。

相較之下，綠色線與黃色線代表不使用 moment matching 的結果，其中綠色線為固定亂數種子但不使用 moment matching，黃色線為不固定亂數種子且不使用 moment matching。這兩條線的波動明顯較大，尤其在中高波動率區間，例如 σ 約 0.4 到 0.55 附近，可以看到曲線出現較明顯的上下跳動。這表示當不使用 moment matching 時，亂數樣本本身的平均數或標準差若偏離理論標準常態分配，就會直接反映在模擬價格上，使估計結果較不穩定。

固定亂數種子與不固定亂數種子的差異，主要反映在結果是否可重現以及曲線細節是否會因不同亂數樣本而改變。固定亂數種子的結果每次執行都會相同，適合用來檢查與比較；不固定亂數種子則會隨著每次執行產生不同亂數樣本，呈現蒙地卡羅模擬的自然抽樣變動。從圖中可以看到，在使用 moment matching 時，固定與不固定亂數種子的差異較小；但在不使用 moment matching 時，兩者差異比較明顯。

整體而言，這張圖說明了兩個重點。第一，不論採用哪一種亂數種子與 moment matching 設定，European call option 的模擬價格都會隨著波動率上升而增加，表示波動率與買權價格的正向關係仍然成立。第二，moment matching 可以改善模擬結果的穩定性，使曲線較平滑、不同亂數種子下的結果較接近；而不使用 moment matching 時，模擬價格較容易受到亂數抽樣誤差影響，曲線波動也會更明顯。



4. 證據

本題的證據主要來自四條模擬曲線的比較。程式延續前一題的設定，並在固定亂數種子與不固定亂數種子的基礎上，加入使用與不使用 moment matching 的兩種情況，因此圖中共有四條線。這樣的設計可以直接觀察亂數種子設定與 moment matching 是否會影響 European call option 模擬價格。

從圖表可以看到，四條曲線雖然在局部位置有高低差異，但整體都隨著波動率 sigma 上升而增加。這證明本題的主要結論仍然成立：不論是否固定亂數種子，也不論是否使用 moment matching，European call option 的價格都會隨波動率提高而上升。

另外，圖中藍色線與紅色線代表使用 moment matching 的結果，兩條線相對接近且較平穩；綠色線與黃色線代表不使用 moment matching 的結果，波動明顯較大。這可以作為 moment matching 有助於降低抽樣誤差的證據。也就是說，moment matching 將亂數樣本校正為平均數接近 0、標準差接近 1 後，模擬結果會比較穩定。

因此，圖表證明了兩點：第一，波動率仍是影響 European call option 價格的重要因素；第二，moment matching 主要影響模擬結果的穩定性，而亂數種子是否固定則影響結果的可重現性與局部波動。整體而言，這些證據回應了本題要比較亂數種子設定與 moment matching 對模擬結果影響的目的。

5. 結論

綜合本題結果可以得知，在固定亂數種子與不固定亂數種子兩種情況下，European call option 的模擬價格都會隨著波動率 σ 增加而上升；同時，不論是否使用 moment matching，整體趨勢也都維持一致。這表示買權價格與波動率之間的正向關係並不會因亂數種子設定或 moment matching 設定而改變，而是由 GBM 模型與買權 payoff 結構本身所決定。

Moment matching 的主要作用，是讓模擬所使用的標準常態亂數更接近理論分配，因此可以降低有限樣本造成的抽樣誤差。從圖表可以看出，使用 moment matching 的曲線較平穩，固定亂數種子與不固定亂數種子之間的差異也較小；相反地，不使用 moment matching 時，曲線波動較明顯，表示模擬價格更容易受到亂數樣本偏差影響。

因此，本題可以得到的結論是：固定亂數種子可以提升結果的可重現性，moment matching 則可以提升模擬結果的穩定性。雖然不同設定下的曲線細節會有所差異，但它們共同支持相同的主要結論，也就是在其他條件固定時，European call option 的價格會隨著波動率上升而增加。

（四）題目：

（c）結果以 streamlit 視覺化互動網頁呈現。

1. 目的

本題的目的，是將子主題 A 中 European call option 與波動率 σ 的模擬分析，進一步製作成 Streamlit 互動式視覺化頁面。透過互動式介面，使用者可以自行調整模型參數與模擬設定，並即時觀察 European call option 模擬價格與波動率之間的變化關係。

此程式特別提供多個可調整設定，包括 σ 範圍、模擬路徑數、亂數種子數

值、固定或不固定亂數種子，以及是否使用 moment matching。這樣做的目的，是讓使用者不只看到單一固定設定下的圖表，而是可以透過調整參數，觀察不同模擬條件如何影響選擇權價格曲線。

因此，本題除了延續前面關於波動率、亂數種子與 moment matching 的分析，也進一步強調互動式視覺化的功能。透過 Streamlit 頁面，可以更直觀地比較不同參數設定下的模擬結果，幫助理解蒙地卡羅模擬中參數選擇與亂數設定對 European call option 價格的影響。

2. 程式

這一段是在建立 Streamlit 互動式網頁所需的初始環境。程式先載入數值運算、統計分配、繪圖以及 Streamlit 相關套件，其中數值運算與統計套件用來進行蒙地卡羅模擬與常態分配轉換，繪圖套件用來產生選擇權價格與波動率的圖表，而 Streamlit 則用來把原本的 Python 模擬程式轉換成可以互動操作的網頁介面。

接著，程式設定 Streamlit 頁面的基本資訊，包括網頁標題與版面配置。版面設定為寬版，是為了讓圖表與參數設定區有較大的顯示空間，使用者在操作時可以更清楚地看到圖形變化與控制項。最後，程式設定圖表的字體大小，使後續輸出的圖表標題、座標軸與圖例更容易閱讀。整體來說，這一段是為 Streamlit 視覺化頁面與後續模擬圖表做準備。

```
1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 import streamlit as st
5
6
7 st.set_page_config(page_title="子主題A 選擇權價格模擬", layout="wide")
8
9 font_size = 12
10 plt.rcParams.update({"font.size": font_size})
```

下一步是建立選擇權定價與蒙地卡羅模擬的核心函數，功能與前面子主題 A 的程式架構相同。第一個函數用來計算 Black-Scholes 理論價格，透過輸入初始股價、履約價格、無風險利率、到期時間、波動率與選擇權型態，得到歐式買權或賣權的理論價值。這個函數可以作為模擬價格的參考基準。

第二個函數則是 European option 的蒙地卡羅模擬函數，用來根據 GBM 模型計算到期股價，再由到期股價計算選擇權 payoff，最後將平均 payoff 折現回現在，得到模擬價格。若使用 moment matching，程式會先將亂數樣本重新標準化，使其平均數接近 0、標準差接近 1，以降低有限樣本下的抽樣誤差。

這兩個函數在 Streamlit 版本中仍然是主要計算基礎。差別在於，前面的 Python 檔案是固定參數後直接執行並輸出圖表；而這裡的 Streamlit 版本會將這些函數接到互動式參數設定上，讓使用者在網頁中改變參數後，程式可以即時重新計算並更新圖表。

```
13  #%%
14  def Black_Scholes(S0, K, r, T, sigma, option_type="call"):
15      d1 = (np.log(S0/K)+(r+sigma**2/2)*T)/(sigma*np.sqrt(T))
16      d2 = d1-sigma*np.sqrt(T)
17      if option_type == "call":
18          option_price = S0*stats.norm.cdf(d1)-K*np.exp(-r*T)*stats.norm.cdf(d2)
19      if option_type == "put":
20          option_price = K*np.exp(-r*T)*stats.norm.cdf(-d2)-S0*stats.norm.cdf(-d1)
21
22      return option_price
23
24  def European_option_simulation(S0, K, r, T, sigma, Z,
25                                moment_matching=False,
26                                option_type="call"):
27      if moment_matching:
28          Z = (Z-np.mean(Z, axis=0))/np.std(Z, axis=0)
29          # 每個 column 計算平均數和標準差，做 moment matching。
30
31      ST = S0*np.exp((r-0.5*sigma**2)*T+sigma*np.sqrt(T)*Z)
32
33      if option_type == "call":
34          payoff = np.maximum(ST-K, 0)
35      if option_type == "put":
36          payoff = np.maximum(K-ST, 0)
37
38      option_prices = np.exp(-r*T)*np.mean(payoff, axis=0)
39
40      return option_prices
```

這一段是在建立 Streamlit 頁面的互動式參數設定區。程式先設定頁面標題，讓使用者知道此頁面是用來觀察 European call option 模擬價格與波動率之間的關係；接著在側邊欄建立參數設定區，讓使用者可以直接調整模型與模擬參數。

在金融模型參數方面，側邊欄提供初始股價、履約價格、無風險利率與到期時間的設定。初始股價與履約價格預設為 100，代表一開始以價平買權作為分析基準；無風險利率預設為 0.02，到期時間預設為 0.5，延續前面子主題 A 的基本設定。這些參數會影響 GBM 模型下的到期股價分布，以及最後計算出的選擇權價格。

接著，程式提供 sigma 範圍與取點數的調整功能。使用者可以設定波動率的起始值與結束值，並決定要在這個範圍內取幾個點來繪製曲線。預設值為 0.1 到 0.6，共取 26 個點，與前面題目的設定一致。這樣可以讓使用者觀察在低波動到高波動情境下，European call option 價格如何變化。

在模擬設定方面，程式將選擇權型態固定為 call，並提供 moment matching 的勾選功能。若勾選使用 moment matching，程式會將亂數樣本重新標準化，使其平

均數接近 0、標準差接近 1，以降低抽樣誤差；若取消勾選，則使用原始亂數樣本進行模擬。這讓使用者可以直接比較使用與不使用 moment matching 時，模擬曲線是否有所不同。

此外，側邊欄也提供模擬路徑數、固定亂數種子與亂數種子數值的設定。模擬路徑數預設為 1000，使用者可以增加或減少模擬樣本數，觀察樣本數對曲線穩定性的影響。固定亂數種子的開關則用來控制結果是否可重現；若固定亂數種子，程式會使用指定的 seed 值；若不固定，則每次執行可能產生不同亂數樣本。整體而言，這一段是 Streamlit 互動功能的核心，讓原本固定的模擬程式變成可以即時調整參數的視覺化工具。

```
43  #%%
44  st.title("子主題A-(c): European call option 模擬價格與波動率")
45
46  st.sidebar.header("參數設定")
47
48  S0 = st.sidebar.slider("初始股價 S0",
49                        min_value=50.0,
50                        max_value=150.0,
51                        value=100.0,
52                        step=1.0)
53  K = st.sidebar.slider("履約價格 K",
54                      min_value=50.0,
55                      max_value=150.0,
56                      value=100.0,
57                      step=1.0)
58  r = st.sidebar.slider("無風險利率 r",
59                      min_value=0.0,
60                      max_value=0.2,
61                      value=0.02,
62                      step=0.01)
63  T = st.sidebar.selectbox("到期時間 T",
64                          options=[0.1, 0.5, 1.0, 2.0, 3.0],
65                          index=1)
66
67  sigma_range = st.sidebar.slider("sigma 範圍",
68                                 min_value=0.01,
69                                 max_value=1.0,
70                                 value=(0.1, 0.6),
71                                 step=0.01)
72  sigma_points = st.sidebar.slider("sigma 取點數",
73                                  min_value=10,
74                                  max_value=100,
75                                  value=26,
76                                  step=1)
```

```
78  option_type = "call"
79  moment_matching = st.sidebar.checkbox("使用 Moment Matching", value=True)
80
81  m_paths = st.sidebar.slider("模擬路徑數 m_paths",
82                              min_value=100,
83                              max_value=10000,
84                              value=1000,
85                              step=100)
86  N = 1
```

```

88     fixed_seed = st.sidebar.checkbox("固定亂數種子", value=True)
89     seed = st.sidebar.number_input("亂數種子 seed",
90                                   min_value=0,
91                                   max_value=999999,
92                                   value=42,
93                                   step=1)
94
95     random_type = "pseudo"

```

這一段是在根據 Streamlit 側邊欄的設定，實際產生波動率資料並計算模擬價格。程式會先依照使用者設定的 sigma 範圍與取點數，建立一組等距的波動率數列。也就是說，使用者在網頁上調整 sigma 的起點、終點或取點數後，這裡就會重新產生對應的波動率資料，後續圖表也會跟著更新。

接著，程式建立兩組結果陣列，分別用來儲存兩種模擬方式下的選擇權價格。第一種是所有波動率共用同一批亂數樣本，第二種是針對每一個波動率重新產生亂數樣本。這樣設計是為了延續子主題 A 的比較方式，讓使用者在互動頁面中也能看到平滑曲線與逐一參數重新抽樣曲線之間的差異。

在亂數設定方面，程式會根據使用者是否勾選固定亂數種子來決定亂數起始狀態。如果使用者選擇固定亂數種子，程式會使用輸入的 seed 數值，使每次執行能得到相同的模擬結果；如果選擇不固定亂數種子，程式則會讓系統自行產生亂數起點，使每次結果可能略有不同。之後，程式會依照設定的亂數產生方式建立標準常態亂數，作為 GBM 模型中的隨機項。

第一部分計算的是共用同一批亂數樣本的模擬價格。程式先產生一批標準常態亂數，然後在不同波動率下都使用這批相同的亂數進行模擬。這樣做可以讓不同波動率之間的差異主要來自 sigma 本身，因此曲線通常較平滑，適合觀察波動率對 European call option 價格的主要影響。

第二部分則是逐一參數重新抽樣的模擬方式，也就是在每一個波動率設定下重新產生一批亂數，再計算該波動率對應的模擬價格。為了讓這組結果也能依照使用者選擇固定或不固定亂數種子，程式在進入這部分前會再次設定亂數狀態。這種方式可以呈有限樣本下蒙地卡羅模擬的抽樣變異，使圖表能同時顯示較平滑的趨勢與較貼近實際重複抽樣情況的估計結果。

```

98     #%%
99     sigmas = np.linspace(sigma_range[0], sigma_range[1], sigma_points)
100
101     option_prices_same_Z_mean = np.zeros(len(sigmas))
102     option_prices_for_loop_mean = np.zeros(len(sigmas))

```

```

104     if fixed_seed:
105         np.random.seed(int(seed))
106     else:
107         np.random.seed(None)
108
109     if random_type == "pseudo":
110         u = np.random.rand(m_paths, N)
111         Z = stats.norm.ppf(u)
112
113     if random_type == "np.random.randn":
114         Z = np.random.randn(m_paths, N)
115
116     if random_type == "np.random.normal":
117         Z = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
118
119     for i in range(len(sigmas)):
120         sigma = sigmas[i]
121
122         option_prices_same_Z = European_option_simulation(S0, K, r, T, sigma, Z,
123                                                         moment_matching=moment_matching,
124                                                         option_type=option_type)
125
126         option_prices_same_Z_mean[i] = np.mean(option_prices_same_Z)

```

```

129     if fixed_seed:
130         np.random.seed(int(seed))
131     else:
132         np.random.seed(None)
133
134     for i in range(len(sigmas)):
135         sigma = sigmas[i]
136
137         if random_type == "pseudo":
138             u_for_loop = np.random.rand(m_paths, N)
139             Z_for_loop = stats.norm.ppf(u_for_loop)
140
141         if random_type == "np.random.randn":
142             Z_for_loop = np.random.randn(m_paths, N)
143
144         if random_type == "np.random.normal":
145             Z_for_loop = np.random.normal(loc=0.0, scale=1.0, size=(m_paths, N))
146
147         option_prices_for_loop = European_option_simulation(S0, K, r, T, sigma, Z_for_loop,
148                                                         moment_matching=moment_matching,
149                                                         option_type=option_type)
150
151         option_prices_for_loop_mean[i] = np.mean(option_prices_for_loop)

```

這一段是將模擬結果輸出到 Streamlit 頁面上。程式先建立三個資訊欄位，分別顯示目前的亂數種子設定、是否使用 moment matching，以及目前選擇的 sigma 範圍。這樣使用者在觀看圖表時，可以同時確認目前圖表是在哪些設定下產生的，避免只看到曲線卻不知道對應的參數條件。

接著，程式建立圖表，並將兩種模擬結果畫在同一張圖中。藍色虛線代表所有波動率共用同一批亂數的結果，曲線通常較平滑；藍色實線則代表逐一參數重新抽樣的結果，也就是每一個波動率下重新產生亂數後得到的模擬價格。透過這兩條線，使用者可以在 Streamlit 頁面中直接比較不同亂數使用方式對模擬曲線的影響。

最後，程式設定圖表的橫軸為波動率 sigma，縱軸為 European call option 價格，並加入標題、圖例與格線，使圖表更容易閱讀。完成圖表設定後，程式將圖表顯示在 Streamlit 網頁中，讓使用者在調整側邊欄參數後，可以即時看到新的模

擬結果。

```
154 #%%
155 col1, col2, col3 = st.columns(3)
156 col1.metric("Seed 設定", "固定" if fixed_seed else "不固定")
157 col2.metric("Moment Matching", "True" if moment_matching else "False")
158 col3.metric("sigma 範圍", f"{sigma_range[0]:.2f} - {sigma_range[1]:.2f}")
159
160 fig = plt.figure(figsize=(10, 6))
161
162 plt.plot(sigmas,
163          option_prices_same_Z_mean,
164          color="blue",
165          linewidth=2,
166          linestyle="--",
167          label="same Z")
168
169 plt.plot(sigmas,
170          option_prices_for_loop_mean,
171          color="blue",
172          linewidth=2,
173          linestyle="-",
174          label="Z in for loop")
175
176 plt.xlabel(r"$\sigma$")
177 plt.ylabel("European call option price")
178 plt.title("European call option simulation price and volatility")
179 plt.legend(loc="upper left", fontsize=8)
180 plt.grid(alpha=0.3)
181 plt.tight_layout()
182 st.pyplot(fig)
```

3. 結果

從第一張圖可以看到，在預設參數下，Streamlit 頁面設定為初始股價為 100、履約價格為 100、無風險利率為 0.02、到期時間為 0.5，並且使用 sigma 範圍 0.10 到 0.60、模擬路徑數 1000、固定亂數種子與 moment matching。這些設定延續前面子主題 A 的基本情境，也就是價平 European call option 的模擬分析。

在這個設定下，圖表顯示 European call option 的模擬價格會隨著波動率 sigma 上升而增加。當 sigma 較低時，買權價格大約在 3 到 4 附近；當 sigma 增加到 0.6 時，價格上升到約 17 左右。這表示在其他條件固定時，波動率提高會增加到期股價大幅上漲的可能性，進而提高買權的期望 payoff，因此選擇權價格上升。

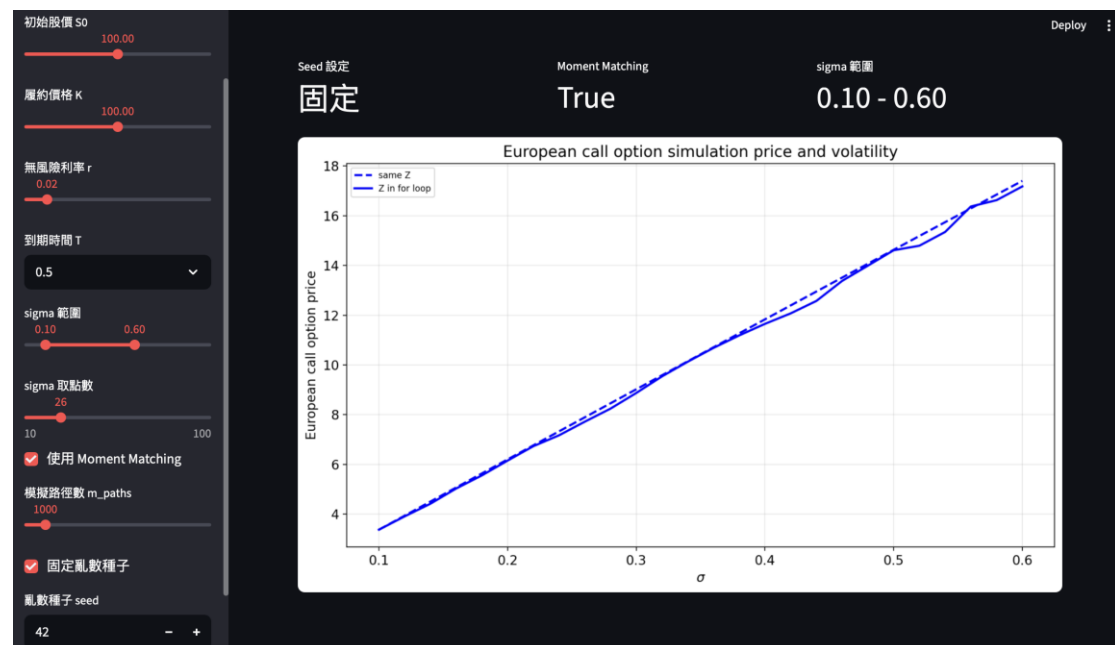
圖中同時有兩條藍色線。虛線代表所有波動率共用同一批亂數樣本，因此不同波動率之間的差異主要來自 sigma 本身，曲線較平滑；實線代表逐一參數重新抽樣，也就是每一個波動率下重新產生亂數，因此會出現較明顯的局部波動。兩條線雖然在細節上略有差異，但整體趨勢一致，表示波動率與買權價格的正向關係相當穩定。

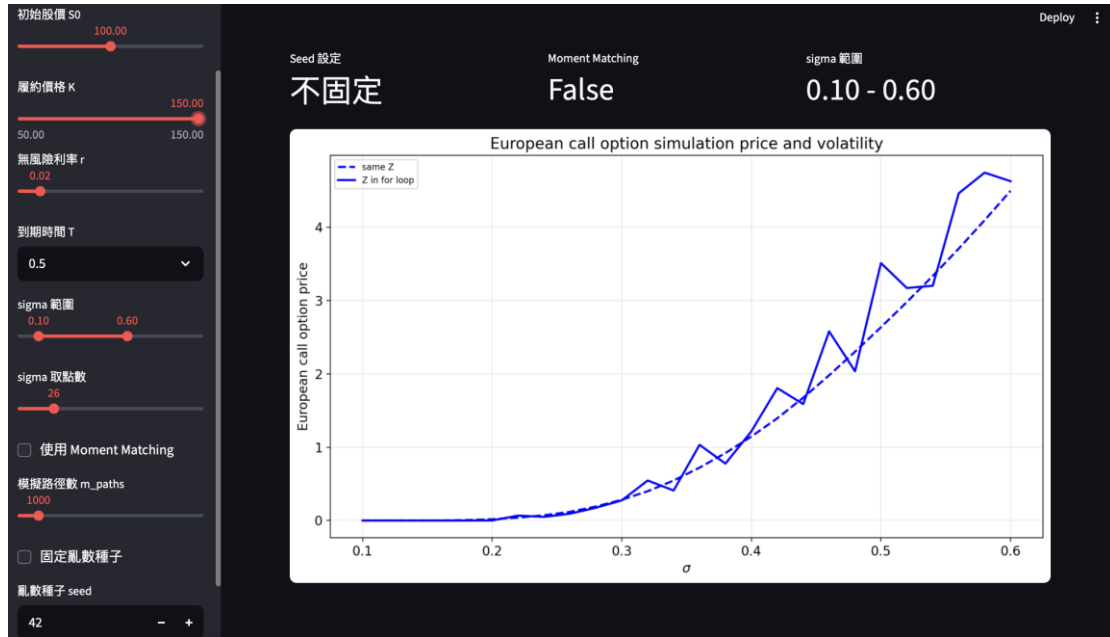
第二張圖則展示了使用者改變參數後，圖表會即時更新的效果。此時履約價格被提高到 150，代表買權變成深度價外；同時不使用 moment matching，且亂數

種子設定為不固定。由於履約價格遠高於初始股價，標的股價必須上漲到很高才會產生正 payoff，所以在低波動率區間，買權價格幾乎接近 0。只有當波動率逐漸提高時，股價大幅上漲並超過履約價格的可能性增加，買權價格才開始明顯上升。

在第二張圖中，實線的波動比第一張圖更明顯，這主要與不固定亂數種子、不使用 moment matching，以及價外買權本身 payoff 較少發生有關。當 K 很高時，只有少數模擬路徑會產生正 payoff，因此模擬價格更容易受到抽樣結果影響；再加上沒有使用 moment matching 來校正亂數樣本，曲線就會呈現較大的上下跳動。這也說明互動式頁面能讓使用者清楚觀察不同參數設定下，模擬結果的穩定程度如何改變。

整體而言，這兩張圖說明 Streamlit 頁面的核心價值：使用者可以透過側邊欄改變股價、履約價格、波動率範圍、亂數種子設定與 moment matching，並即時觀察圖表變化。預設情況下，圖表清楚呈現 European call option 價格隨波動率上升而增加；當參數改變為深度價外、不固定亂數種子且不使用 moment matching 時，圖表則呈現較低價格與較大抽樣波動。這表示 Streamlit 不只是輸出固定圖表，而是能用互動方式幫助理解不同模型參數與模擬設定對選擇權價格的影響。





4. 證據

本題的證據來自 Streamlit 頁面在不同設定下即時產生的圖表。程式的目的，是將前面 European call option 與波動率的模擬分析做成互動式網頁；從頁面中可以看到，使用者確實可以透過側邊欄調整初始股價、履約價格、無風險利率、到期時間、`sigma` 範圍、模擬路徑數、亂數種子設定與 moment matching，並讓圖表即時更新。

在步驟上，程式先根據使用者設定產生波動率數列，再分別計算共用同一批亂數與逐一參數重新抽樣的模擬價格，最後將兩條曲線畫在同一張圖中。第一張圖保留預設設定，包含 $S_0 = 100$ 、 $K = 100$ 、固定亂數種子與使用 moment matching。結果顯示兩條曲線皆隨著 sigma 增加而上升，證明在互動式頁面中仍能呈現前面子題得到的主要結論。

第二張圖則提供了參數改變後的證據。當履約價格提高到 $K = 150$ 、不固定亂數種子且不使用 moment matching 時，買權價格在低波動率區間接近 0，直到波動率提高後才明顯上升，而且曲線波動也變得更明顯。這說明 Streamlit 頁面不只是固定呈現單一圖表，而是能根據不同參數設定反映不同模擬結果。

因此，從兩張圖可以證明本題成功達成互動式視覺化目的：使用者調整參數後，模擬結果會即時改變；同時，圖表也能清楚呈現波動率、履約價格、亂數種子設定與 moment matching 對 European call option 模擬價格的影響。

5. 結論

綜合本題結果可以得知，Streamlit 互動式頁面能有效延伸前面子主題 A 的分析，讓 European call option 的蒙地卡羅模擬不再只是固定參數下的一張圖，而是可以透過使用者調整參數即時重新計算與更新圖表。透過側邊欄設定，使用者可以直接改變波動率範圍、履約價格、模擬路徑數、亂數種子是否固定，以及是否使用 moment matching，進一步觀察不同設定對模擬價格的影響。

從預設情況可以看出，當其他參數固定時，European call option 的價格會隨著波動率 σ 上升而增加，這與前面子題的結論一致。當履約價格提高、不固定亂數種子且不使用 moment matching 時，圖表則呈現較低的買權價格與較明顯的抽樣波動，說明模擬結果會受到履約價格、亂數樣本與 moment matching 設定影響。

因此，本題的結論是，Streamlit 視覺化可以更直觀地呈現蒙地卡羅模擬結果，並幫助使用者理解不同參數對 European call option 價格的影響。它不僅驗證了波動率與買權價格之間的正向關係，也展示了互動式工具在金融計算分析中的便利性與解釋效果。

伍、總結

題目：(d) 加入完整結論。

子主題 A 主要是在假設股價服從 GBM 的情況下，使用蒙地卡羅模擬方法探討 European call option 模擬價格與波動率 σ 之間的關係，並進一步比較不同亂數種子設定、是否使用 moment matching，以及 Streamlit 互動式視覺化下的模擬結果。整體而言，本子主題的核心目標，是了解波動率、亂數抽樣方式與模擬校正方法如何影響買權價格。

從基本模擬結果可以看出，在其他參數固定時，European call option 的價格會隨著波動率 σ 增加而上升。這是因為買權 payoff 具有不對稱性：當到期股價低於履約價時，買權可以不履約，payoff 最低為 0；但當股價高於履約價時，payoff 會隨股價上升而增加。因此，波動率越高，雖然股價下跌的可能性也提高，但股價大幅上漲帶來的潛在 payoff 也增加，使買權價格提高。

在 for 迴圈寫法的比較中，可以發現共用同一批亂數時，曲線較平滑，較適合觀察波動率對選擇權價格的主要影響；而逐一參數重新抽樣時，每個波動率都使用新的亂數樣本，因此曲線較容易出現局部波動。這說明蒙地卡羅模擬的結果除了受到金融參數影響，也會受到亂數抽樣方式影響。不過，兩種寫法得到的整體趨勢一致，都支持買權價格隨波動率上升而增加。

在固定亂數種子與不固定亂數種子的比較中，結果顯示兩者的曲線雖然在局部位置可能有些微差異，但整體走勢非常接近。固定亂數種子的優點是結果具有可重現性，適合用於報告分析與程式檢查；不固定亂數種子則能呈現不同亂數樣本下蒙地卡羅模擬的自然變動。因此，亂數種子設定主要影響的是結果是否可重現與曲線細節，而不會改變波動率與買權價格之間的基本正向關係。

在 moment matching 的比較中，可以看出使用 moment matching 後，模擬結果通常較穩定，固定亂數種子與不固定亂數種子的差異也較小。原因是 moment matching 會將標準常態亂數重新校正，使樣本平均數接近 0、標準差接近 1，降低有限樣本造成的抽樣誤差。相反地，不使用 moment matching 時，模擬價格較容易受到亂數樣本偏差影響，曲線波動也較明顯。因此，moment matching 的主要作用是提升蒙地卡羅模擬結果的穩定性。

最後，Streamlit 互動式頁面進一步將子主題 A 的模擬分析視覺化。透過側邊欄調整波動率範圍、履約價格、模擬路徑數、亂數種子設定與 moment matching，使用者可以即時觀察不同參數下的模擬價格變化。預設設定下，圖表仍清楚呈現買權價格隨波動率上升而增加；當調整為較高履約價、不固定亂數種子或不使用 moment matching 時，圖表則會反映出價格下降或曲線波動增加的情形，顯示互動式工具能有效幫助理解參數變化對選擇權價格的影響。

綜合而言，子主題 A 的結果顯示，波動率是影響 European call option 價格的重要因素，且兩者呈現明顯正向關係。亂數種子設定會影響模擬結果的可重現性，逐一參數重新抽樣會呈現蒙地卡羅抽樣波動，而 moment matching 則能提升模擬穩定性。透過程式模擬與 Streamlit 視覺化，本子主題不僅驗證了選擇權價格與波動率之間的金融直覺，也說明蒙地卡羅方法在選擇權定價與敏感度分析中的應用價值。